Lab 11: Government Data APIs

INFO 4617/5617 · Week 11 · Chapter 11, Government Data APIs

This week every class is live coding: one lab in three parts, one a day. We use two public APIs, each with an idea that holds for APIs in general. On Monday and Wednesday, the **Census Bureau** and *what can I ask for?*: from a table on its website to the same number from its API, then many places at once, and whether to trust a number. On Friday, **FRED**, the Federal Reserve Bank of St. Louis's data service, and *let the server compute*. The chapter's third API, the EIA's, is for later, on your own (Step 14 says where).

Each code cell holds a comment that says what to write; cells marked `# Given` are already written, and we read and run them together. After each step, a 💬 **Discuss** box holds questions to talk over; click it to open it. Short ⏱️ microsprints ask you to change one thing, alone or in pairs.

Before class: open this notebook in the `webdata` environment (chapter 1), and have your email address ready for the User-Agent. Get two free keys (Get Your Keys), and store each as an environment variable, as the chapter's "Setting Environment Variables" shows:

- `CENSUS_KEY`, from the Census Bureau, for Monday and Wednesday; click the activation link in its email before you use it;
- `FRED_KEY`, from FRED, for Friday; FRED asks you to make a free account first, so allow a few minutes.

The simplest way is one line per key in a terminal (on Windows, the Anaconda Prompt), `conda env config vars set CENSUS_KEY="your-key" -n webdata`. Then **quit Jupyter** (restarting the kernel isn't enough), close the terminal, open a new one, run `conda activate webdata`, and start Jupyter from it. Treat a key like a password: never paste it into a cell or a code file.

On Wednesday and Friday, start at that day's setup step.

> **Instructor key.** Rebuilt from scratch on 2026-11-10 for the next offering, after the audit in issue #167 and the rules `labs/AUTHORING.md` took that day; the key taught this term was not edited into shape. It follows chapter 11 at textbook commit `3036573`, without the EIA (the instructor's decision of 2026-10-28; Step 14 points to it in the chapter). Run on 2026-11-10 with `labs/tools/run_key.py` from the course's cloud machine, three times, and every cell that didn't change between runs gave byte-for-byte the same outputs, figures included: every code cell ran, none is tagged `not-run-live`, and the `getpass()` cells did nothing, because both keys were set. The Census requests use the fixed ACS 2020–2024 five-year estimates, and every FRED request has a fixed window, so a rerun should match these outputs unless FRED revises a series, or October 2025's missing CPI value is filled in. Microsprints ran on the key's choices: "median gross rent", `B25034`, `county:13`, and Step 6's five places. The four checkpoints in `labs/week-11/data/` were saved the same day by the key's own code, run once more with a `to_csv()` line after each of the four cells that make them. Their loading cells are commented out, so `run_key.py` didn't run them; each was run separately against the local files. `checkpoints` points at `main`, where the files exist once this lab's pull request merges.

Part 1 — Monday: the Census, and what can I ask for?

Every number on the Census Bureau's website also comes from its API. We start from one table on the website, find it in the API's own catalog, and ask the API for the website's number, by hand. Then we break the request on purpose, to see how the API says no.

Step 0 — Set up, and check your key (6 minutes)

An API key identifies you, so treat it like a password: keep it out of every cell and file, and read it from an **environment variable**, a setting Jupyter receives when it starts (Reading the Keys in

Python). Put your email in the first given cell and run it. If it says `MISSING`, quit Jupyter and start it from a new terminal (When a Key Says `MISSING`), or run the next cell, which asks with `getpass()`.

```
# Given: imports; your User-Agent (put your email in it); a check that CENSUS_KEY is set, not the
 ↪  key
import os
import time
import requests
import pandas as pd
import matplotlib.pyplot as plt

pd.set_option("display.max_colwidth", None)   # show whole descriptions, not "..."
HEADERS = {"User-Agent": "WebDataScience/1.0 (your-email@colorado.edu)"}
# saved copies, for when a cell stops:
checkpoints =
 ↪  "https://raw.githubusercontent.com/cuinfoscience/INFO4617-Fall2026/main/labs/week-11/data/"
print("CENSUS_KEY:", "set" if os.environ.get("CENSUS_KEY") else "MISSING")
```

```
CENSUS_KEY: set
```

```
# Given: only if the check said MISSING. A box opens: paste your key there and press Enter
# (nothing shows as you paste). If the check said set, this does nothing
from getpass import getpass
if not os.environ.get("CENSUS_KEY"):
    os.environ["CENSUS_KEY"] = getpass("CENSUS_KEY isn't set. Paste it here: ")
```

```
# census_key: read "CENSUS_KEY" from os.environ with square brackets (never print it)
census_key = os.environ["CENSUS_KEY"]
```

💬 Discuss

- In week 10 we told Wikipedia who we are with a User-Agent. What can a key do that a User-Agent can't?
- The Census Bureau takes the key as a parameter in the URL, so `response.url` holds it. Where could a printed URL end up, and could you take it back?
- Why doesn't a key set after Jupyter started reach the notebook, even after you restart the kernel?

Step 1 — From the website to the catalog (11 minutes)

On data.census.gov, table **B19013** gives Boulder County's median household income: $103,994, ±$2,268, in the American Community Survey's (ACS) five-year estimates for 2020–2024 (figure 11.1). The API calls a table a **group**, and lists every group in a catalog, `groups.json`. This is our first request to the API, so print what came back before reading it. Then search the descriptions with `str.contains()`, as in "From the Website's Table to the API's Group".

```
# census_base: "https://api.census.gov/data/2024/acs/acs5", the ACS 5-year estimates for
 ↪  2020-2024
census_base = "https://api.census.gov/data/2024/acs/acs5"
```

```
# groups_response: request census_base + "/groups.json" with headers=HEADERS (the catalog needs
 ↪  no key)
# Print its status code and the first 200 characters of its text
groups_response = requests.get(census_base + "/groups.json", headers=HEADERS)
print(groups_response.status_code, groups_response.text[:200])
```

```
200 {
  "groups": [
```

```
    {
      "name": "B17015",
      "description": "Poverty Status in the Past 12 Months of Families by Family Type by Social
       ↪  Security Income by Supplemental Security Income (SSI) and
```

```
# catalog: the response's .json(); show its keys
catalog = groups_response.json()
catalog.keys()
```

```
dict_keys(['groups'])
```

```
# groups: a DataFrame of catalog["groups"], keeping its "name" and "description"; print how many;
 ↪  show 3
groups = pd.DataFrame(catalog["groups"])
groups = groups[["name", "description"]]
print(len(groups), "groups")
groups.head(3)
```

```
1199 groups
```

```
     name  \
0  B17015
1  B18104
2  B17016
```

```
0  Poverty Status in the Past 12 Months of Families by Family Type by Social Security Income by
 ↪  Supplemental Security Income (SSI) and Cash Public Assistance Income
1
 ↪  Sex by Age by Cognitive Difficulty
2                                                  Poverty Status in the Past 12 Months of
 ↪  Families by Family Type by Work Experience of Householder and Spouse
```

```
# Given, only if the cells above stopped: remove the "#" from the next line, and run this cell.
# It loads the catalog saved on 2026-11-10 by the key's code (1,199 groups, as the live request
 ↪  returns)
# groups = pd.read_csv(checkpoints + "lab11_groups.csv")
```

```
# match: True where groups["description"] contains "median household income" (case=False)
# Show groups[match], sorted by "name"
match = groups["description"].str.contains("median household income", case=False)
groups[match].sort_values("name")
```

```
         name  \
1087   B19013
568   B19013A
560   B19013B
563   B19013C
```

```
574   B19013D
576   B19013E
570   B19013F
572   B19013G
532   B19013H
534   B19013I
1084   B19019
1193   B19049
1119   B22008
70     B25099
987    B25119
211    B29004
```

```
1087                                                                Median Household Income in
 ↪  the Past 12 Months (in 2024 Inflation-Adjusted Dollars)
568                                       Median Household Income in the Past 12 Months (in 2024
 ↪  Inflation-Adjusted Dollars) (White Alone Householder)
560                     Median Household Income in the Past 12 Months (in 2024 Inflation-Adjusted
 ↪  Dollars) (Black or African American Alone Householder)
563            Median Household Income in the Past 12 Months (in 2024 Inflation-Adjusted Dollars)
 ↪  (American Indian and Alaska Native Alone Householder)
574                                       Median Household Income in the Past 12 Months (in 2024
 ↪  Inflation-Adjusted Dollars) (Asian Alone Householder)
576   Median Household Income in the Past 12 Months (in 2024 Inflation-Adjusted Dollars) (Native
 ↪  Hawaiian and Other Pacific Islander Alone Householder)
570                               Median Household Income in the Past 12 Months (in 2024
 ↪  Inflation-Adjusted Dollars) (Some Other Race Alone Householder)
572                                    Median Household Income in the Past 12 Months (in 2024
 ↪  Inflation-Adjusted Dollars) (Two or More Races Householder)
532                Median Household Income in the Past 12 Months (in 2024 Inflation-Adjusted
 ↪  Dollars) (White Alone, Not Hispanic or Latino Householder)
534                                  Median Household Income in the Past 12 Months (in 2024
 ↪  Inflation-Adjusted Dollars) (Hispanic or Latino Householder)
1084                                                 Median Household Income in the Past 12 Months
 ↪  (in 2024 Inflation-Adjusted Dollars) by Household Size
1193                                            Median Household Income in the Past 12 Months (in
 ↪  2024 Inflation-Adjusted Dollars) by Age of Householder
1119            Median Household Income in the Past 12 Months (in 2024 Inflation-Adjusted
 ↪  Dollars) by Receipt of Food Stamps/SNAP in the Past 12 Months
70                                                 Mortgage Status by Median Household Income in
 ↪  the Past 12 Months (in 2024 Inflation-Adjusted Dollars)
987                                                          Median Household Income in the Past 12
 ↪  Months (in 2024 Inflation-Adjusted Dollars) by Tenure
```

```
211                                  Median Household Income for Households With a Citizen,
 ↪  Voting-Age Householder (in 2024 Inflation-Adjusted Dollars)
```

```
# Given: the search as a function. find_groups("internet") shows the groups whose description has
 ↪  those words
# (regex=False reads your words as they are, parentheses included)
def find_groups(words):
    match = groups["description"].str.contains(words, case=False, regex=False)
    return groups[match].sort_values("name")
```

⏱️ Microsprint (4 minutes, alone or in pairs)

A table for a question of your own. `find_groups()` is a **function**: a name for the search above, which runs it on whatever words you put between its parentheses.

Do: search for a topic of your own, such as ”median gross rent”, ”travel time to work”, or ”internet”. The key searches for ”median gross rent”. **Share:** your words, how many groups matched, and the one you'd use.

```
# Microsprint: find_groups("your words")
find_groups("median gross rent")
```

```
       name  \
234  B25031
361  B25064
344  B25071
994  B25111
997  B25113

                                                                                description
234                                                                 Median Gross Rent by Bedrooms
361                                                                   Median Gross Rent (Dollars)
344  Median Gross Rent as a Percentage of Household Income in the Past 12 Months (Dollars)
994                                                 Median Gross Rent by Year Structure Built
997                                 Median Gross Rent by Year Householder Moved Into Unit
```

💬 Discuss

- `B19013` came back with nine siblings, `B19013A` to `B19013I`. What do the letters add, and when would you want one of them instead?
- The website says *table* and the API says *group*. What else did you translate from the website's words to the API's?
- The lab saves a copy of this catalog in case your request fails. What can a saved copy not tell you that the live request can?

Step 2 — A group's variables (9 minutes)

A group holds **variables**, the columns of the website's table, listed at `groups/B19013.json` (as a page). A name is the group, a line and a letter: `B19013_001E` is line 1's **E**stimate, `B19013_001M` its **M**argin of error, the website's ”±”; `EA` and `MA` are annotations. Go down the JSON one level a cell, as in ”From the Group to Its Variables”. The given `show_estimates()` repeats it for any group: its `for` loop runs the indented lines once for each name.

```
# b19013_response: request census_base + "/groups/B19013.json"; b19013: its .json(); show its
 ↪  keys
b19013_response = requests.get(census_base + "/groups/B19013.json", headers=HEADERS)
b19013 = b19013_response.json()
b19013.keys()
```

```
dict_keys(['variables'])
```

```
# group_vars: b19013["variables"]; show its names with sorted()
group_vars = b19013["variables"]
sorted(group_vars)
```

```
['B19013_001E',
 'B19013_001EA',
 'B19013_001M',
 'B19013_001MA',
 'GEO_ID',
 'NAME']
```

```
# Show what the catalog says about "B19013_001E"
group_vars["B19013_001E"]
```

```
{'label': 'Estimate!!Median household income in the past 12 months (in 2024 inflation-adjusted
 ↪  dollars)',
 'concept': 'Median Household Income in the Past 12 Months (in 2024 Inflation-Adjusted Dollars)',
 'predicateType': 'int',
 'group': 'B19013',
 'limit': 0,
 'predicateOnly': True,
 'universe': 'Households'}
```

```
# Given: show_estimates(group) asks for a group's catalog and prints each estimate, a name ending
 ↪  in "E", and
# its label. NAME ends in "E" too, but it's the place's name, not an estimate, so it's skipped
def show_estimates(group):
    response = requests.get(census_base + "/groups/" + group + ".json", headers=HEADERS)
    variables = response.json()["variables"]
    for name in sorted(variables):
        if name.endswith("E") and name != "NAME":
            print(name, "|", variables[name]["label"])

show_estimates("B19013")
```

```
B19013_001E | Estimate!!Median household income in the past 12 months (in 2024 inflation-adjusted
 ↪  dollars)
```

⏱️ Microsprint (3 minutes, alone or in pairs)

A table with more lines: group `B25034`, Year Structure Built, which we use on Wednesday. The `!!` in a label separates its levels, the way the website indents a table's rows.

Do: run `show_estimates()` on `"B25034"`. **Share:** how many estimates it has, and which line counts homes built in the 1970s.

```
# Microsprint: show_estimates("B25034")
show_estimates("B25034")
```

```
B25034_001E | Estimate!!Total:
B25034_002E | Estimate!!Total:!!Built 2020 or later
B25034_003E | Estimate!!Total:!!Built 2010 to 2019
B25034_004E | Estimate!!Total:!!Built 2000 to 2009
B25034_005E | Estimate!!Total:!!Built 1990 to 1999
B25034_006E | Estimate!!Total:!!Built 1980 to 1989
B25034_007E | Estimate!!Total:!!Built 1970 to 1979
B25034_008E | Estimate!!Total:!!Built 1960 to 1969
```

```
B25034_009E | Estimate!!Total:!!Built 1950 to 1959
B25034_010E | Estimate!!Total:!!Built 1940 to 1949
B25034_011E | Estimate!!Total:!!Built 1939 or earlier
```

💬 Discuss

- The website shows an estimate and its margin side by side; the API makes them two variables. Why might an API split them?
- `show_estimates()` skips `NAME`, though its name ends in `E`. Without that test, how many lines would B25034 seem to have?
- Which variables would you ask for to rebuild the website's table exactly?

Step 3 — The website's number, from the API (11 minutes)

A request names the variables with `get`, the place with `for`, what that place sits inside with `in`, and you with `key` (the API's user guide). Places are FIPS codes, strings of digits: Colorado is state `08`, and Boulder County is county `013` within it. The answer is a list of lists: row 0 holds the column names, rows 1 onward (`rows[1:]`) the places, and every value is a string. See "The Same Numbers from the API".

```
# Given: the request's parameters, a dictionary. Its keys are in quotes:
# for and in are words Python keeps for itself
county_params = {
    "get": "NAME,B19013_001E,B19013_001M",   # the place's name, the estimate, and its margin of
    ↪  error
    "for": "county:013",                     # Boulder County...
    "in": "state:08",                        # ...in Colorado
    "key": census_key,
}
```

```
# county_response: requests.get() census_base with params=county_params and HEADERS; print
↪  status, text[:200]
county_response = requests.get(census_base, params=county_params, headers=HEADERS)
print(county_response.status_code, county_response.text[:200])
```

```
200 [["NAME","B19013_001E","B19013_001M","state","county"],
["Boulder County, Colorado","103994","2268","08","013"]]
```

```
# county_rows: the response's .json(); show it
county_rows = county_response.json()
county_rows
```

```
[['NAME', 'B19013_001E', 'B19013_001M', 'state', 'county'],
 ['Boulder County, Colorado', '103994', '2268', '08', '013']]
```

```
# county: a DataFrame of county_rows[1:], with county_rows[0] as its columns; show it
county = pd.DataFrame(county_rows[1:], columns=county_rows[0])
county
```

	NAME	B19013_001E	B19013_001M	state	county
0	Boulder County, Colorado	103994	2268	08	013

```
# Given, only if the cells above stopped: remove the "#" from the next line, and run this cell.
# It loads the row saved on 2026-11-10 by the key's code; dtype=str keeps "08" and "013" as text
# county = pd.read_csv(checkpoints + "lab11_county.csv", dtype=str)
```

```
# Turn the columns "B19013_001E" and "B19013_001M" into numbers with pd.to_numeric; show county
county["B19013_001E"] = pd.to_numeric(county["B19013_001E"])
```

```
county["B19013_001M"] = pd.to_numeric(county["B19013_001M"])
county
```

```
                      NAME  B19013_001E  B19013_001M state county
0  Boulder County, Colorado       103994         2268    08    013
```

💬 Discuss

- Did the API's number match the website's, $103,994 ± $2,268? If it hadn't, what would you check first: the dataset's year, the variable, or the place?
- Why must `08` stay a string? What would `int("08")` lose?
- In week 7, the Wayback Machine's CDX API answered in the same shape: a list of lists whose first row is the header. Why might an API send rows that way rather than as a list of dictionaries?

Step 4 — A city, a state, and when the API says no (13 minutes)

The city of Boulder is place `07850`. Ask for it without `in`, and read the answer: a status of 400 comes with the Bureau's reason in words. A place needs its state, because place codes are numbered within each state. The given `look()` holds the two lines you typed; add `in` and look again. Then a given loop asks for a state, a county and a city, as in "The Same Variable for a State, a County, and a City".

```
# Given: the city of Boulder, without "in"
city_params = {"get": "NAME,B19013_001E,B19013_001M", "for": "place:07850", "key": census_key}
```

```
# city_response: send city_params to census_base; print its status code and text[:200]
city_response = requests.get(census_base, params=city_params, headers=HEADERS)
print(city_response.status_code, city_response.text[:200])
```

```
400 error: ambiguous geography "place:07850". You must either specify a wildcard or full qualify
 ↪  it with state
```

```
# Given: the two lines you just typed, as a function. look(params) sends params and prints what
 ↪  came back
def look(params):
    response = requests.get(census_base, params=params, headers=HEADERS)
    print(response.status_code, response.text[:200])
```

```
# Add "in": set city_params["in"] to "state:08", then look() at city_params
city_params["in"] = "state:08"
look(city_params)
```

```
200 [["NAME","B19013_001E","B19013_001M","state","place"],
["Boulder city, Colorado","87493","6221","08","07850"]]
```

```
# Given: the same two variables for a state, a county, and a city. Only "for" and "in" change.
# .update() adds one dictionary's keys to another; [1][:3] is row 1's first three values;
# time.sleep(1) pauses a second between requests
geographies = [
    {"for": "state:08"},                          # Colorado
    {"for": "county:013", "in": "state:08"},      # Boulder County, in Colorado
    {"for": "place:07850", "in": "state:08"},     # the city of Boulder, in Colorado
]
three = []
for geography in geographies:
    geo_params = {"get": "NAME,B19013_001E,B19013_001M", "key": census_key}
    geo_params.update(geography)
    geo_rows = requests.get(census_base, params=geo_params, headers=HEADERS).json()
    three.append(geo_rows[1][:3])
```

```
    time.sleep(1)
pd.DataFrame(three, columns=["NAME", "B19013_001E", "B19013_001M"])
```

```
                      NAME B19013_001E B19013_001M
0                 Colorado       95470         578
1  Boulder County, Colorado      103994        2268
2    Boulder city, Colorado       87493        6221
```

```
# Given: try_params, a fresh copy of Step 3's parameters, for the microsprint. Run this again to
 ↪  start over
try_params = {"get": "NAME,B19013_001E,B19013_001M", "for": "county:013", "in": "state:08",
              "key": census_key}
```

⏱️ Microsprint (5 minutes, alone or in pairs)

Break it on purpose. Each mistake below makes Step 3's `.json()` stop with the same error, `JSONDecodeError`; what came back before it says which mistake it was.

Do: make one mistake in `try_params`, and `look()` at it:

- drop a zero from a code: `try_params["for"] = "county:13"` (the key's choice);
- misspell the variable: `try_params["get"] = "NAME,B19013_01E";`
- put a space after the comma: `try_params["get"] = "NAME, B19013_001E";`
- put the key's name in quotes: `try_params["key"] = "census_key".`

Share: your mistake, the status code, and the first words of the text.

```
# Microsprint: change one value in try_params (see the list above), then look(try_params)
try_params["for"] = "county:13"
look(try_params)
```

```
204
```

💬 Discuss

- Across the room, a mistake came back three ways: a 400 with a reason, a 204 with nothing at all, and a 200 with a web page for a bad key. In week 5 we read status codes: why isn't checking for 200 enough here?
- None of our cells handles an error; the chapter's code calls `raise_for_status()`, which stops on a 4xx or 5xx status. Which of today's mistakes would it catch, and which would get past it?
- The city's margin of error is about ten times the state's. Why does a smaller place get a wider margin?

Part 2 — Wednesday: many places, and whether to trust a number

One request can ask for several places, or for every place in a state. Then two questions before trusting a number: is a difference bigger than its margin of error, and is the number a number at all?

Step 5 — Set up again (4 minutes)

Part 2 starts fresh, so a student who missed Monday can start here: nothing from Monday's cells is needed. Put your email in the first given cell, and run both. They import what we need, check and read `CENSUS_KEY` (Step 0 says what to do if it says `MISSING`), and set `census_base`, the address of the ACS 5-year estimates for 2020–2024.

```
# Given: imports; your User-Agent (put your email in it); a check that CENSUS_KEY is set, not the
 ↪  key
import os
import requests
import pandas as pd
import matplotlib.pyplot as plt

HEADERS = {"User-Agent": "WebDataScience/1.0 (your-email@colorado.edu)"}
checkpoints =
 ↪  "https://raw.githubusercontent.com/cuinfoscience/INFO4617-Fall2026/main/labs/week-11/data/"
census_base = "https://api.census.gov/data/2024/acs/acs5"
print("CENSUS_KEY:", "set" if os.environ.get("CENSUS_KEY") else "MISSING")
```

```
CENSUS_KEY: set
```

```
# Given: ask for the key with getpass only if it's MISSING (Step 0), then read it
from getpass import getpass
if not os.environ.get("CENSUS_KEY"):
    os.environ["CENSUS_KEY"] = getpass("CENSUS_KEY isn't set. Paste it here: ")
census_key = os.environ["CENSUS_KEY"]
```

💬 Discuss

- Did your key work today? If it didn't on Monday, which cause in the chapter's "When a Key Says `MISSING`" was it?

Step 6 — Several places in one request (16 minutes)

Put several codes in `for`, separated by commas, and one request returns a row for each: `place:07850,45970` is Boulder and Longmont, as in "Comparing Places". Monday's cells are gathered in a given function, `census_get()`: it sends the parameters, prints the status, makes the DataFrame, and turns each variable's column (each name starting with `B`) into numbers. `def` names the lines, and `return` hands back the result. The given chart, the chapter's figure 11.4, uses it for ten variables at once.

```
# Given: Monday's by-hand cells, as one function. census_get(params) returns the answer as a
 ↪  DataFrame
def census_get(params):
    response = requests.get(census_base, params=params, headers=HEADERS)
    print("status", response.status_code)
    rows = response.json()
    table = pd.DataFrame(rows[1:], columns=rows[0])
    for column in table.columns:
        if column.startswith("B"):
            table[column] = pd.to_numeric(table[column])
    return table

pair_params = {"get": "NAME,B19013_001E,B19013_001M", "for": "place:07850", "in": "state:08",
               "key": census_key}
```

```
# Set pair_params["for"] to Boulder and Longmont: "place:07850,45970"
pair_params["for"] = "place:07850,45970"
```

```
# incomes: census_get(pair_params); show it
incomes = census_get(pair_params)
incomes
```

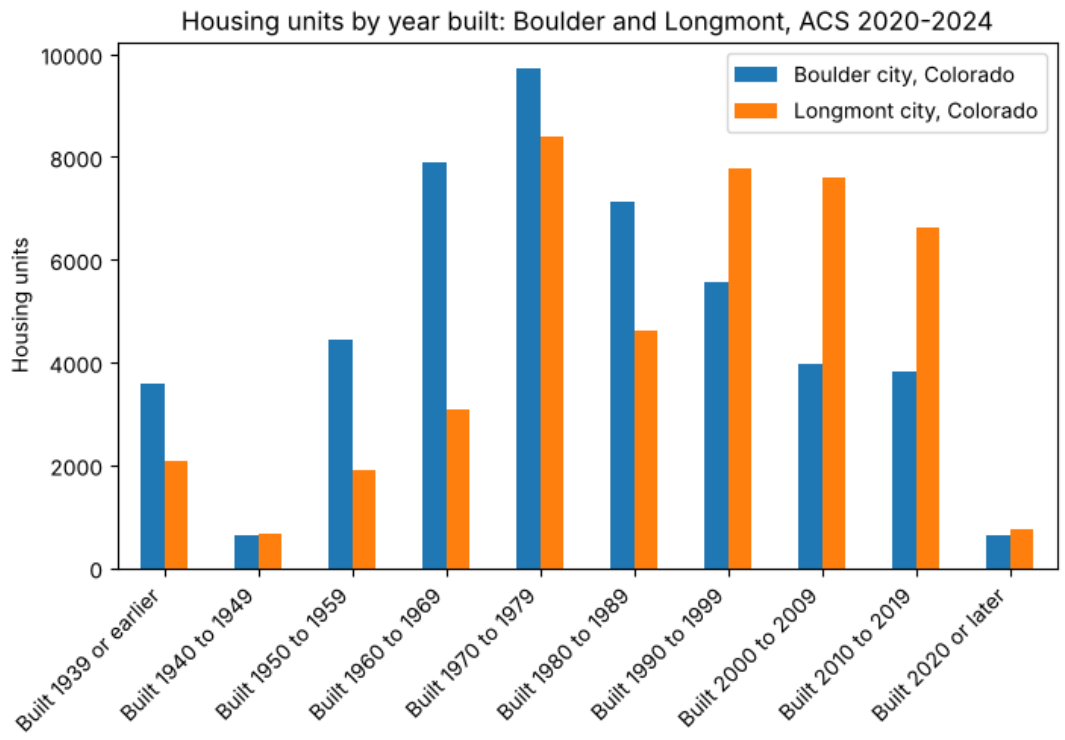
```
status 200
```

	NAME	B19013_001E	B19013_001M	state	place
0	Boulder city, Colorado	87493	6221	08	07850
1	Longmont city, Colorado	90671	2975	08	45970

```python
# Given: the chapter's figure 11.4. housing_vars, a list comprehension, writes the ten codes
# B25034_002E to _011E; labels keeps each label's last part, after its last "!!";
# .T turns the table, and iloc[::-1] puts the oldest decade first
housing_vars = [f"B25034_{line:03d}E" for line in range(2, 12)]
housing_params = {"get": "NAME," + ",".join(housing_vars), "for": "place:07850,45970", "in":
 ↪  "state:08",
                  "key": census_key}
df_compare = census_get(housing_params)
b25034 = requests.get(census_base + "/groups/B25034.json", headers=HEADERS).json()["variables"]
labels = {var: b25034[var]["label"].split("!!")[-1] for var in housing_vars}
housing = df_compare.set_index("NAME")[housing_vars].rename(columns=labels).T

fig, ax = plt.subplots(figsize=(8, 4.5))
housing.iloc[::-1].plot.bar(ax=ax)
ax.set(title="Housing units by year built: Boulder and Longmont, ACS 2020-2024",
       ylabel="Housing units", xlabel="")
plt.setp(ax.get_xticklabels(), rotation=45, ha="right")
ax.legend(title="")
plt.show()
```

```
status 200
```




```python
# Given: for five places, all housing units (B25034_001E) and those built since 2000 (_002E to
 ↪  _004E):
# Boulder, Longmont, Fort Collins (27425), Lakewood (43000), and Castle Rock (12415)
five_params = {"get": "NAME,B25034_001E,B25034_002E,B25034_003E,B25034_004E",
               "for": "place:07850,45970,27425,43000,12415", "in": "state:08", "key": census_key}
```

```
five = census_get(five_params)
five["since_2000"] = five["B25034_002E"] + five["B25034_003E"] + five["B25034_004E"]
five
```

```
status 200
```

	NAME	B25034_001E	B25034_002E	B25034_003E	B25034_004E	state	place	since_2000
0	Boulder city, Colorado	47465	646	3836	3968	08	07850	8450
1	Castle Rock town, Colorado	29159	2412	8989	9352	08	12415	20753
2	Fort Collins city, Colorado	74558	1619	11019	14600	08	27425	27238
3	Lakewood city, Colorado	73547	1373	6841	5208	08	43000	13422
4	Longmont city, Colorado	43528	761	6631	7597	08	45970	14989

⏱️ Microsprint (5 minutes, alone or in pairs)

A short version of Additional Exercise 1. Cities differ in size, so counts don't compare fairly; shares do.

Do: divide each place's units built since 2000 by its total, `B25034_001E`. **Share:** the place with the largest share, the one with the smallest, and the two shares.

```
# Microsprint: five["share"]: five["since_2000"] divided by five["B25034_001E"]; show NAME and
 ↪  share
five["share"] = five["since_2000"] / five["B25034_001E"]
five[["NAME", "share"]]
```

	NAME	share
0	Boulder city, Colorado	0.178026
1	Castle Rock town, Colorado	0.711719
2	Fort Collins city, Colorado	0.365326
3	Lakewood city, Colorado	0.182496
4	Longmont city, Colorado	0.344353

💬 Discuss

- Longmont has more units than Boulder from every decade since the 1990s. The chapter names one likely reason, Boulder's 1976 cap on building permits. What would you need to know to say it was the reason?
- The chart counts the units standing in 2020–2024 by the year they were built. What does it leave out?
- `census_get()` has no error handling. If the API answered with a 429 or a maintenance page, which of its lines would stop, and what would the error say?

Step 7 — Is the difference real? (10 minutes)

Longmont's median household income is higher than Boulder's, but each is an estimate with a margin of error, at 90% confidence. Don't judge by whether the two ranges overlap: the ACS handbook says that test isn't reliable (chapter 7, p. 55). Compare the difference with its own margin: square the two margins (`** 2`), add them, and take the square root (`** 0.5`). A difference smaller than its margin may be no difference at all.

```
# estimates: incomes["B19013_001E"]; difference: Longmont's (row 1) minus Boulder's (row 0);
 ↪  print it
estimates = incomes["B19013_001E"]
difference = estimates[1] - estimates[0]
print(difference)
```

```
3178
```

```
# margins: incomes["B19013_001M"]; margin_of_difference: (margins[0] ** 2 + margins[1] ** 2) **
 ↪  0.5; print it
margins = incomes["B19013_001M"]
margin_of_difference = (margins[0] ** 2 + margins[1] ** 2) ** 0.5
print(round(margin_of_difference))
```

```
6896
```

💬 Discuss

- The difference is smaller than its margin. What can you say about which city's households earn more, and what can't you?
- Boulder's margin is about twice Longmont's. What about the city could make its estimate less certain?
- Which checks like this one could a pipeline run on its own, every time it runs?

Step 8 — Every place, and numbers that aren't (20 minutes)

A wildcard, `place:*` with `in` set to `state:08`, asks for every place in Colorado: its cities, towns, and census-designated places. Ask for median gross rent, group `B25064`, and `describe()` the column before you trust it. Where too few households answered, the API sends a code, `-666666666`, not a rent; the Bureau's notes list each code. `mask()` turns those rows into `NaN`, a missing value, as in "Every Place in a State, and a Number That Isn't".

```
# Given: median gross rent (B25064_001E) and its margin of error, for the city of Boulder
rent_params = {"get": "NAME,B25064_001E,B25064_001M", "for": "place:07850", "in": "state:08",
               "key": census_key}
```

```
# Set rent_params["for"] to every place, "place:*"; places: census_get(rent_params); print how
 ↪  many rows
rent_params["for"] = "place:*"
places = census_get(rent_params)
print(len(places), "places")
```

```
status 200
482 places
```

```
# Given, only if the cell above stopped: remove the "#" from the next line, and run this cell.
# It loads the 482 places saved on 2026-11-10 by the key's code, with the place codes kept as
 ↪  text
# places = pd.read_csv(checkpoints + "lab11_places.csv", dtype={"state": str, "place": str})
```

```
# describe() the column "B25064_001E", rounded with .round(): look at its min and its max
places["B25064_001E"].describe().round()
```

```
count          482.0
mean    -235130433.0
std      318871284.0
min     -666666666.0
25%     -666666666.0
```

```
50%            976.0
75%           1663.0
max           3501.0
Name: B25064_001E, dtype: float64
```

```
# no_estimate: True where "B25064_001E" equals -666666666; print how many places have no estimate
no_estimate = places["B25064_001E"] == -666666666
print(no_estimate.sum(), "of", len(places), "places have no estimate")
```

```
170 of 482 places have no estimate
```

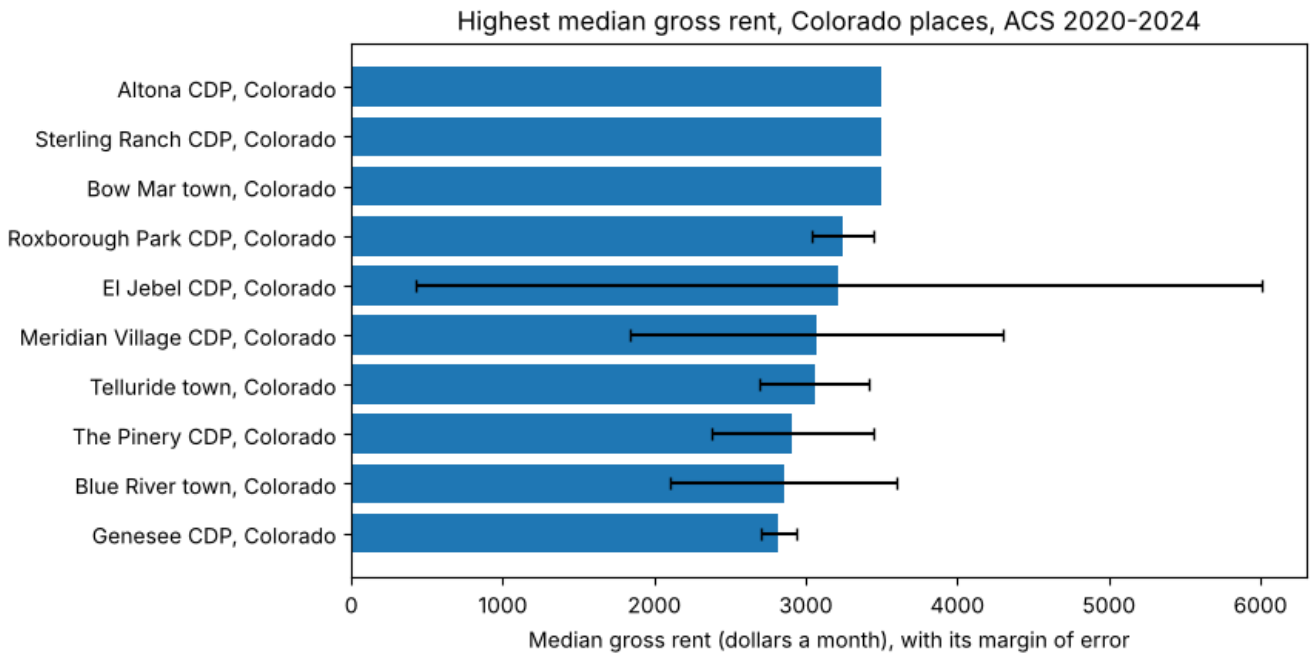
```
# places["rent"]: "B25064_001E" with the no_estimate rows masked; describe() it, rounded
places["rent"] = places["B25064_001E"].mask(no_estimate)
places["rent"].describe().round()
```

```
count     312.0
mean     1488.0
std       596.0
min       330.0
25%      1008.0
50%      1415.0
75%      1867.0
max      3501.0
Name: rent, dtype: float64
```

```
# top: places sorted by "rent" (ascending=False), its first 10 rows; show NAME, rent, B25064_001M
top = places.sort_values("rent", ascending=False).head(10)
top[["NAME", "rent", "B25064_001M"]]
```

	NAME	rent	B25064_001M
10	Altona CDP, Colorado	3501.0	-333333333
421	Sterling Ranch CDP, Colorado	3501.0	-333333333
46	Bow Mar town, Colorado	3501.0	-333333333
380	Roxborough Park CDP, Colorado	3244.0	205
139	El Jebel CDP, Colorado	3216.0	2790
293	Meridian Village CDP, Colorado	3071.0	1233
434	Telluride town, Colorado	3056.0	359
435	The Pinery CDP, Colorado	2910.0	535
39	Blue River town, Colorado	2853.0	747
176	Genesee CDP, Colorado	2819.0	118

```
# Given: the ten as horizontal bars, each with a line as wide as its margin of error on either
 ↪  side.
# A margin below zero is a code too, so it's masked; invert_yaxis() puts the highest at the top
top_margin = top["B25064_001M"].mask(top["B25064_001M"] < 0)
fig, ax = plt.subplots(figsize=(8, 4.5))
ax.barh(top["NAME"], top["rent"], xerr=top_margin, color="tab:blue", ecolor="black", capsize=3)
ax.invert_yaxis()
ax.set(title="Highest median gross rent, Colorado places, ACS 2020-2024",
       xlabel="Median gross rent (dollars a month), with its margin of error", ylabel="")
plt.show()
```



⏱️ Microsprint (4 minutes, alone or in pairs)

The margin column has a code of its own.

Do: find the places whose margin, `B25064_001M`, is `-333333333`, and look the code up in the Bureau's notes. **Share:** how many places, their median rent, and what the code says about it.

```
# Microsprint: capped: True where "B25064_001M" equals -333333333; show places[capped]
capped = places["B25064_001M"] == -333333333
places[capped]
```

```
                           NAME  B25064_001E  B25064_001M state  place  \
10           Altona CDP, Colorado         3501   -333333333    08  01740
46         Bow Mar town, Colorado         3501   -333333333    08  08070
421  Sterling Ranch CDP, Colorado         3501   -333333333    08  73943

       rent
10   3501.0
46   3501.0
421  3501.0
```

💬 Discuss

- More than a third of Colorado's places have no estimate of median gross rent. Left in, what would `-666666666` have done to a mean, and to a ranking?
- The chart's top three are capped at 3,501, "$3,500 or more", and El Jebel's margin is ±$2,790. Which of the ten could you rank with confidence?
- Typed without `in`, `county:*` returns every county in the country (3,222 when this lab was written), with no error. How would you notice before your chart did?

Part 3 — Friday: FRED, and let the server compute

FRED (Federal Reserve Economic Data), from the Federal Reserve Bank of St. Louis, holds about 800,000 economic series, and it can transform one before it sends it: a change, a percent change, a

lower frequency. Asking the server, instead of downloading everything and doing it in pandas, is today's idea, and so is checking it once.

Step 9 — Set up again (3 minutes)

Part 3 starts fresh and needs nothing from Monday or Wednesday. Put your email in the first given cell, and run both: they import what we need and check `FRED_KEY` (Step 0 says what to do if it says `MISSING`). Then read the key.

```
# Given: imports; your User-Agent (put your email in it); a check that FRED_KEY is set, not the
 ↪  key
import os
import requests
import pandas as pd
import matplotlib.pyplot as plt

HEADERS = {"User-Agent": "WebDataScience/1.0 (your-email@colorado.edu)"}
checkpoints =
 ↪  "https://raw.githubusercontent.com/cuinfoscience/INFO4617-Fall2026/main/labs/week-11/data/"
print("FRED_KEY:", "set" if os.environ.get("FRED_KEY") else "MISSING")
```

```
FRED_KEY: set
```

```
# Given: only if the check said MISSING, ask for the key with getpass (Step 0). Otherwise it does
 ↪  nothing
from getpass import getpass
if not os.environ.get("FRED_KEY"):
    os.environ["FRED_KEY"] = getpass("FRED_KEY isn't set. Paste it here: ")
```

```
# fred_key: read "FRED_KEY" from os.environ with square brackets (never print it)
fred_key = os.environ["FRED_KEY"]
```

💬 Discuss

- The Census Bureau's key went in a parameter named `key`; FRED's is named `api_key`. What should you read before you send a key to a new API?

Step 10 — A series, and what it is (11 minutes)

FRED names each series with an ID, shown on its web page (figure 11.2): Boulder County's median household income is `MHICO08013A052NCEN`. First ask the `series` endpoint what it is. FRED reports a mistake as JSON with an `error_message`, so print that before reading on. Then ask `series/observations` for the values. FRED sends each value as a string, and a missing one as `"."`, which `pd.to_numeric(errors="coerce")` turns into `NaN`. See "Finding Series IDs".

```
# fred_base: "https://api.stlouisfed.org/fred"
fred_base = "https://api.stlouisfed.org/fred"
```

```
# Given: which series, your key, and JSON (without "file_type", FRED answers in XML)
series_params = {"series_id": "MHICO08013A052NCEN", "api_key": fred_key, "file_type": "json"}
```

```
# series_url: fred_base + "/series"; series_response: send series_params to it; series_json: its
 ↪  .json()
# Print the status code and series_json.get("error_message"), which is None when there's no error
series_url = fred_base + "/series"
series_response = requests.get(series_url, params=series_params, headers=HEADERS)
series_json = series_response.json()
print(series_response.status_code, series_json.get("error_message"))
```

```
200 None
```

```
# about: series_json["seriess"][0], the one series; print its "title", "frequency" and "units"
about = series_json["seriess"][0]
print(about["title"], "|", about["frequency"], "|", about["units"])
```

```
Estimate of Median Household Income for Boulder County, CO | Annual | Dollars
```

```
# Given: the same series, up to a fixed end, so a rerun gets the same answer
income_params = {"series_id": "MHICO08013A052NCEN", "api_key": fred_key, "file_type": "json",
                 "observation_end": "2024-12-31"}
```

```
# obs_url: fred_base + "/series/observations"; income_response: send income_params to it
obs_url = fred_base + "/series/observations"
income_response = requests.get(obs_url, params=income_params, headers=HEADERS)
```

```
# income_json: the response's .json(); income: a DataFrame of income_json["observations"]
# Keep its "date" and "value" columns; show the first 8 rows
income_json = income_response.json()
income = pd.DataFrame(income_json["observations"])
income = income[["date", "value"]]
income.head(8)
```

```
         date  value
0  1989-01-01  34200
1  1990-01-01      .
2  1991-01-01      .
3  1992-01-01      .
4  1993-01-01  41676
5  1994-01-01      .
6  1995-01-01  46199
7  1996-01-01      .
```

```
# Convert "value" with pd.to_numeric(..., errors="coerce") and "date" with pd.to_datetime; show
↪  the last 3 rows
income["value"] = pd.to_numeric(income["value"], errors="coerce")
income["date"] = pd.to_datetime(income["date"])
income.tail(3)
```

```
         date     value
33 2022-01-01   96584.0
34 2023-01-01   97170.0
35 2024-01-01  103293.0
```

💬 Discuss

- The Census API answered with a list of lists; FRED with a dictionary whose `observations` is a list of dictionaries. Which turned into a DataFrame more easily, and why?
- FRED's 2024 figure for Boulder County is $103,293; Monday's ACS figure for 2020–2024 was $103,994. The chapter says they come from different programs. Which would you cite, and how would you say which it is?
- Why fix `observation_end` at 2024-12-31, when the series ends in 2024 anyway?

Step 11 — Let the server compute: `units` (6 minutes)

We'll ask for observations several more times, changing one parameter each time, so the given `fred_observations()` gathers Step 10's lines, as the chapter's does. `**params` collects any other parameters you name, such as `units="chg"`, and sends them along. `units` asks FRED to transform the

values first: `chg` is the change from the previous observation, and `pch` the percent change. See "Let the Server Compute: `units`".

```
# Given: Step 10's lines, as the chapter's function. Any parameter you name after the ID goes
↪ into params
def fred_observations(series_id, **params):
    params.update(series_id=series_id, api_key=fred_key, file_type="json")
    response = requests.get(fred_base + "/series/observations", params=params, headers=HEADERS)
    df = pd.DataFrame(response.json()["observations"])[["date", "value"]]
    df["date"] = pd.to_datetime(df["date"])
    df["value"] = pd.to_numeric(df["value"], errors="coerce")
    return df
```

```
# change: fred_observations() for "MHICO08013A052NCEN" with units="chg" and
↪ observation_end="2024-12-31"
change = fred_observations("MHICO08013A052NCEN", units="chg",
                           observation_end="2024-12-31")
```

```
# Given: the percent change too, and the three answers side by side, one row a year
growth = fred_observations("MHICO08013A052NCEN", units="pch", observation_end="2024-12-31")
views = pd.DataFrame({"year": income["date"].dt.year, "dollars": income["value"],
                      "change": change["value"], "percent change": growth["value"]})
views.head(10).round(1)
```

```
   year  dollars  change  percent change
0  1989  34200.0     NaN             NaN
1  1990      NaN     NaN             NaN
2  1991      NaN     NaN             NaN
3  1992      NaN     NaN             NaN
4  1993  41676.0     NaN             NaN
5  1994      NaN     NaN             NaN
6  1995  46199.0     NaN             NaN
7  1996      NaN     NaN             NaN
8  1997  50245.0     NaN             NaN
9  1998  54503.0  4258.0             8.5
```

⏱️ Microsprint (3 minutes, alone or in pairs)

Read what the server computed.

Do: find the years in which Boulder County's median household income *fell*: the rows of `views` whose `change` is below zero. **Share:** how many years, which ones, and the largest fall in dollars.

```
# Microsprint: fell: True where views["change"] is below zero; show views[fell]
fell = views["change"] < 0
views[fell].round(1)
```

```
    year  dollars  change  percent change
12  2001  57079.0 -1522.0            -2.6
13  2002  55810.0 -1269.0            -2.2
20  2009  63757.0 -3003.0            -4.5
21  2010  62215.0 -1542.0            -2.4
23  2012  66894.0 -1207.0            -1.8
31  2020  88289.0   -52.0            -0.1
```

💬 Discuss

- `change` is `NaN` for 1993, 1995 and 1997, though they have dollars. Why, and could income have fallen in a year our list of falls doesn't show?

- FRED computed `change` from the same dollars we have. When is it worth asking the server for something you could compute yourself?
- If FRED answered with an error, which line of `fred_observations()` would stop, and would its error say what FRED said?

Step 12 — Let the server compute: `frequency` (9 minutes)

A series' **frequency** is how often it's measured, and FRED can lower it before sending: `frequency="a"` with `aggregation_method="avg"` averages each year's months. Take `CPIAUCNS`, the national Consumer Price Index (CPI), which is monthly, over a fixed window set by `observation_start` and `observation_end`. Look for a missing month before you trust a yearly average. See "Let the Server Compute: `frequency` and `aggregation_method`".

```
# start: "2015-01-01"; end: "2025-12-31"
start = "2015-01-01"
end = "2025-12-31"
```

```
# monthly: fred_observations() for "CPIAUCNS" with observation_start=start and
↪  observation_end=end; print len()
monthly = fred_observations("CPIAUCNS", observation_start=start, observation_end=end)
print(len(monthly), "months")
```

```
132 months
```

```
# Given, only if the cell above stopped: remove the "#" from the next line, and run this cell.
# It loads the 132 months saved on 2026-11-10 by the key's code, already converted
# monthly = pd.read_csv(checkpoints + "lab11_cpi_monthly.csv", parse_dates=["date"])
```

```
# missing: True where monthly["value"] is NaN (.isna()); show monthly[missing]
missing = monthly["value"].isna()
monthly[missing]
```

```
          date  value
129 2025-10-01    NaN
```

```
# annual: the same, with frequency="a" and aggregation_method="avg" as well; show its last 3 rows
annual = fred_observations("CPIAUCNS", frequency="a", aggregation_method="avg",
                           observation_start=start, observation_end=end)
annual.tail(3)
```

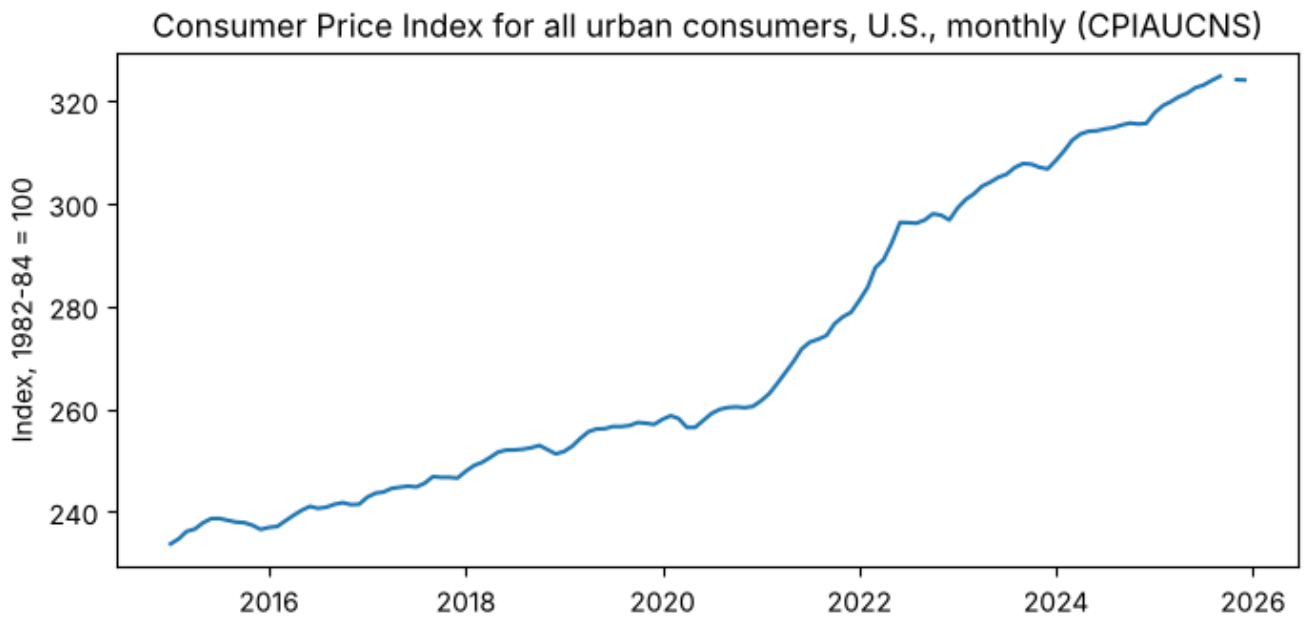
```
         date    value
8  2023-01-01  304.702
9  2024-01-01  313.689
10 2025-01-01  321.943
```

```
# Given: with units="pc1" too, the percent change from the year before: each year's inflation.
↪  FRED applies
# the frequency first, then the units
inflation = fred_observations("CPIAUCNS", frequency="a", aggregation_method="avg", units="pc1",
                              observation_start=start, observation_end=end)
table = pd.DataFrame({"year": annual["date"].dt.year, "mean index": annual["value"],
                      "inflation (%)": inflation["value"]})
table.round(1)
```

```
    year  mean index  inflation (%)
0   2015       237.0            0.1
1   2016       240.0            1.3
2   2017       245.1            2.1
```

```
3   2018       251.1            2.4
4   2019       255.7            1.8
5   2020       258.8            1.2
6   2021       271.0            4.7
7   2022       292.7            8.0
8   2023       304.7            4.1
9   2024       313.7            2.9
10  2025       321.9            2.6
```

```
# Given: the monthly series over time. A NaN leaves a gap in the line
fig, ax = plt.subplots(figsize=(8, 3.5))
ax.plot(monthly["date"], monthly["value"])
ax.set(title="Consumer Price Index for all urban consumers, U.S., monthly (CPIAUCNS)",
       ylabel="Index, 1982-84 = 100", xlabel="")
plt.show()
```



⏱️ Microsprint (3 minutes, alone or in pairs)

Trust the server, but check it once.

Do: from `monthly`, compute each year's mean and count of values with `groupby()` on the year, `monthly["date"].dt.year`, and compare the means with `table`. **Share:** your 2024 and 2025 means beside FRED's, and how many months each is the mean of.

```
# Microsprint: by_year: monthly grouped by monthly["date"].dt.year, its "value" column
# Print the last 3 rows of by_year.mean(), rounded to 1 place, and of by_year.count()
by_year = monthly.groupby(monthly["date"].dt.year)["value"]
print(by_year.mean().tail(3).round(1))
print(by_year.count().tail(3))
```

```
date
2023    304.7
2024    313.7
2025    321.9
Name: value, dtype: float64
date
2023    12
```

```
2024    12
2025    11
Name: value, dtype: int64
```

💬 Discuss

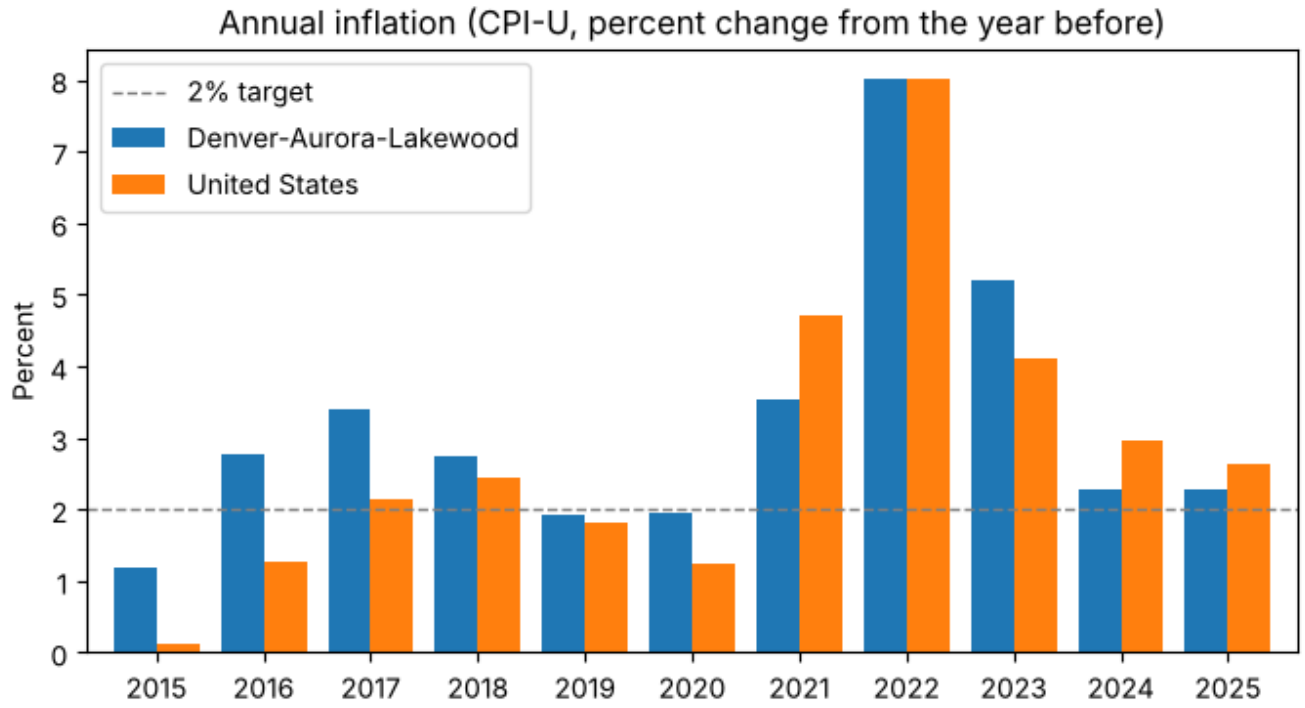
- FRED's 2025 mean and ours agree. How many months is each the mean of? Would you call 2.6 percent ”2025's inflation” without a footnote?
- `aggregation_method` could also be `sum` or `eop`, the end of the period. Which suits a price index, and which a count, like jobs added?
- Misspelled as `observation_begin`, `observation_start` is ignored without a word: FRED sends every month since 1913 (1,356 when this lab was written). How would you notice?

Step 13 — Denver against the nation (3 minutes)

With one series understood, comparing two is asking each the same question. The Bureau of Labor Statistics publishes a CPI for the Denver area, which FRED carries as `CUUSA433SA0`. It's annual already, so it needs only `units="pc1"`. The given cell draws the chapter's figure 11.5: one pair of bars a year, with a dashed line at 2 percent, the Federal Reserve's target.

```
# Given: Denver's inflation, merged with the nation's on the date, as paired bars
denver = fred_observations("CUUSA433SA0", units="pc1", observation_start=start,
 ↪  observation_end=end)
cpi = denver.merge(inflation, on="date", suffixes=("_denver", "_national"))
pairs = cpi.set_index(cpi["date"].dt.year)[["value_denver", "value_national"]]
pairs.columns = ["Denver-Aurora-Lakewood", "United States"]

fig, ax = plt.subplots(figsize=(8, 4))
pairs.plot.bar(ax=ax, rot=0, width=0.8)
ax.axhline(y=2, color="gray", linestyle="--", linewidth=1, label="2% target")
ax.set(title="Annual inflation (CPI-U, percent change from the year before)", ylabel="Percent",
 ↪  xlabel="")
ax.legend()
plt.show()
```



💬 **Discuss**

- In which years did Denver's prices rise faster than the nation's, and in which slower?
- Denver's series is annual and the nation's monthly. What would have gone wrong had we merged the monthly series, without `frequency="a"`?

Step 14 — What did we learn? (3 minutes)

In a few sentences, as comments:

- Step 3 asked the API for the website's number. Did they match, and what would you check first if they hadn't?
- On Monday, a mistake came back three ways. Which is hardest to notice, and what habit catches it?
- In Step 8, how many of Colorado's places had no estimate of median gross rent, and what did `describe()` show before you masked them?
- Which of this week's numbers would be different if you ran this notebook next week, and which wouldn't? Why?

Recommended Exercises ahead (in the chapter's companion notebook, due Sunday):

- Step 1, load your keys without printing them: Step 0's check and `os.environ[...]` (the exercise asks you to print each key's length; our check printed `set`).
- Step 2, one ACS variable for every Colorado county: Steps 1–2 to find the group and the variable, Step 3's `for` and `in`, and Step 8's wildcard, which asked for every place.
- Step 3, build the DataFrame, convert, sort, and print the extremes: Step 3's DataFrame and `pd.to_numeric`, and Step 8's `describe()`, codes, `mask()` and `sort_values()`.
- Step 4, a horizontal bar chart of the top 20 counties: Step 8's sort, `head()` and `barh()` chart.
- Step 5, a related FRED series for Colorado over time: Steps 10–12's `fred_observations()`, `observation_start`, and the line plot of `monthly`; find the series on FRED's website, as Step 10 did.
- Step 6, interpret: the Discuss questions after Steps 3, 7, 8 and 12.

Further, on your own (not graded): the chapter's third API, the U.S. Energy Information Administration's, in "EIA API v2": routes that describe themselves, filters applied on the server, and

paging through a large answer. "Combining Census and EIA Data" then joins Census incomes with the EIA's electricity bills, state by state, through a crosswalk. Its key, `EIA_KEY`, comes from the EIA.

```
# Your answers here, as comments
# 1. They matched: Boulder County came back as 103994 with a margin of 2268, the website's
 ↪  $103,994 and ±$2,268.
#    If they hadn't, I'd check the dataset's year first (2024/acs/acs5 is the 2020-2024
 ↪  estimates), then the
#    variable (B19013_001E, not B19013A to B19013I, and E for the estimate), then the codes
 ↪  (county:013 in state:08).
# 2. The 204: "county:13" came back with status 204 and no text at all, so there's nothing to
 ↪  read, and .json() stops
#    with the same JSONDecodeError as every other mistake. A bad key is close behind: status 200,
 ↪  but a web page.
#    The habit is to print the status and the first 200 characters before calling .json().
# 3. 170 of 482 places had no estimate. Before masking, describe() showed a minimum of
 ↪  -666666666, a 25% quantile
#    of -666666666 too, a median of 976 and a mean of -235130433; masked, 312 places, a median of
 ↪  1415, and a
#    maximum of 3501, which is itself a cap ("$3,500 or more").
# 4. The ACS 2020-2024 estimates are fixed, and every FRED request has a fixed window, so all of
 ↪  them should match
#    unless FRED revises a series. The catalog won't change either: a new ACS release gets a new
 ↪  address
#    (2025/acs/acs5 didn't exist on 2026-11-10), so the groups we found stay the same.
```

Friday: your portfolio checkpoint (15 minutes)

The last 15 minutes go to your project. This week's checkpoint, **Set up your project repo**, is due Sunday, November 1:

- **a public repository** on your own GitHub account, with a descriptive name;
- **`README.md`**, with a title, your name, a one-paragraph summary of your research question, and placeholder sections: Data, Methods, Findings, How to reproduce;
- **`proposal.md`**: your proposal, revised if you've had feedback;
- **placeholders for the rest**: a notebook (`.ipynb`), a `report.md` (or a note that your report will be a PDF), and a `data/` folder.

Use the time to make the repository and push its first commit, here, where you can ask. If your project uses an API key, this week's rule holds there too: the key stays in an environment variable, never in the repository.